

Update: Assuming that this is OK, this will be your project. Marking this as a Check. Alexis is doing more or less the same project. Be sure to read through my notes on Doc2Query on Moodle carefully (under Unit 9).

Hello again, Gabriel,

You asked about a project idea. This is the one I mentioned yesterday. Please let me know if this will work. Thanks!

Prof. Allen

Mini-RAG: Centre Student Handbook Q&A (solo mini-capstone idea for DSC 360)

Goal (MVP)

[Minimum Viable Product — a tiny, working proof-of-concept]

Answer a handful of common questions from the [Student Handbook](#) with a short, accurate snippet and a source citation; say “insufficient context” when you can’t find it.

What you’ll build (end-to-end)

1. **Pick content (Fri):** choose 6–8 relevant sections (e.g., alcohol policy, housing/quiet hours, academic honesty, absences, Title IX contacts, campus safety). Save each section to plain text (one `.txt` per section).
2. **Chunk + metadata:** split by headings; keep chunks under ~700–900 words. For every chunk, store: `{section_title, url, last_updated_if_known, chunk_id, text}` in a simple JSONL or CSV.
3. **Doc2Query questions:** for each chunk, write 3–5 student-style questions that the chunk can answer (e.g., “How many unexcused absences are allowed?” “Who do I contact for Title IX?”). Include one variant that uses the section keyword (e.g., “attendance policy ... absences”). Save: `{question, chunk_id}`.
4. **Embed + index:** embed all generated questions (not the chunks) with a small model (e.g., `all-MiniLM-L6-v2`) and save vectors plus the question→chunk map. A NumPy array + JSONL/CSV is fine.
5. **REPL test loop:** a tiny CLI that
 - embeds the user query,
 - finds top-k similar generated questions,
 - pulls the linked chunk(s),
 - returns a 1–2 sentence answer strictly from the chunk plus a citation (section title + URL). If confidence is low, say “insufficient context.”

6. **Evaluation:** write 10 gold questions up front; check the correct chunk is in top-3. For numeric/policy answers, the returned text must show the exact number/phrase.

Important guardrails

- Always show the section title and URL for the quoted snippet.
- If top-k similarities are weak, say “insufficient context.”
- Keep scope to those 6–8 sections; don’t try to cover the entire handbook.

What to turn in

- `data/`: your `.txt` files
- `index/`: `chunks.jsonl`, `questions.jsonl`, `embeddings.npy` (or `.pkl`)
- `src/`: `ingest.py` (makes chunks), `index.py` (makes questions + embeddings), `ask.py` (REPL)
- `README.md`: 1-page run instructions + your 10 gold Q/A with pass/fail marks

Timeline (two weeks with Thanksgiving break)

- **Week 1 (Nov 14–21)**: collect sections, make chunks + metadata; generate questions; embed; basic REPL working with answers + citations.
- **Thanksgiving Break (Nov 24–28)**: no work expected.
- **Week 2 (Dec 1–5)**: tighten retrieval, finish evaluation on your 10 questions, polish README and demo script. Formal report. **Final deliverables due Fri, Dec 5 at 4:00 PM**. Oral report during the exam slot.

Notes on metadata

Store `section_title` and `url` in every chunk, and include the section keyword in at least one generated question per chunk—this small touch often fixes policy-style queries. Plain cosine similarity on question embeddings is enough for MVP.

[Earlier feedback]

Hello, Gabriel,

We built a simple chatbot in Lab 1, but as we mentioned today in passing, a chatbot alone isn’t yet a project—why wouldn’t a user just use ChatGPT? For the project, the LLM-powered app needs a specific user, a concrete task, and a small, legal dataset (“world”) that justifies building something custom. Right now that’s too vague to greenlight.

Please resubmit with: a working title; 3–5 sentences on who the user is and what task they’ll accomplish; the exact corpus you’ll use and how you’ll get it; the 3–4 question types you’ll support; and what a successful demo looks like (answers show sources and say “insufficient context” when unsure).

Two viable directions you could pick:

- RAG assistant for one small, permitted “world” (e.g., one board game rulebook + FAQ or the Centre student handbook). Answer only a few policy-style questions with citations; decline outside scope.
- Tiny agent with one allow-listed tool (e.g., calculator or a single known API) to complete a clearly defined task end-to-end, with guardrails and a minimal UI (e.g., buttons for “show sources” and “reset”).

If you go the “world” route, add chunk metadata (title/section/topic) and consider a lightweight question-generation index so queries like “grading breakdown” or “excused absences” match reliably. Keep scope small enough to finish solo in two weeks.

Come see me to refine this before you rebuild the proposal.

Prof. Allen